Laxmi Charitable Trust's

Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce

Department of Information Technology (Bsc.IT Semester II)

Scala for Data Science Practical

Practical - 21/22/23/24

Roll No. : S006	Name:Seja Manohar Deshmukh
Class :SYIT	Batch:01
Date of Assignment :18/08/2026	Date/Time of Submission :18/08/2026

Practical 21

Join two CSV files in Spark DataFrames based on a common column and write the output to a file.

INPUT:

```scala
import org.apache.spark.sql.SparkSession

object Prac21 extends App {
  val s = SparkSession.builder()
    .appName("JoinTwoCSV")
    .master("local[*]")
    .getOrCreate()

  val d1 = s.read
    .option("header", "true")
    .csv("File1.csv")

  val d2 = s.read
    .option("header", "true")
    .csv("File2.csv")

  val j = d1.join(d2, Seq("id"), "inner")

  j.show()
  j.write
    .option("header", "true")
    .csv {
      "output"
    }
  s.stop()
}
```

OUTPUT:

```
+---+-----+-----+
| id| name|score|
+---+-----+-----+
|  1|Rohit|   90|
|  2|Suraj|   67|
|  3|Priya|   89|
|  4|Sejal|   50|
+---+-----+-----+
```

PRACTICAL 22

Set up Apache Spark locally and count the frequency of words in a text file.

```scala
import org.apache.spark.sql.SparkSession

object Prac22 extends App {
  val s = SparkSession.builder()
    .appName("WordCountExample")
    .master("local[*]")
    .getOrCreate()

  val r = s.sparkContext.textFile("input.txt")
  val w = r.flatMap(line => line.split("\\s+"))
  val wo = w.map(word => (word, 1))
  val wc = wo.reduceByKey(_ + _)
  wc.collect().foreach(println)

  s.stop()
}
```

```
spark is easy
spark is powerful
```

Output:

```
(is,2)
(easy,1)
(powerful,1)
(spark,2)
```

Practical 23

Filter rows in a csv file using spark dataframes where a numeric column exceeds a threshold.

Scala -> open in -> path popup -> scala (in file manager)-> copy csv file in scala

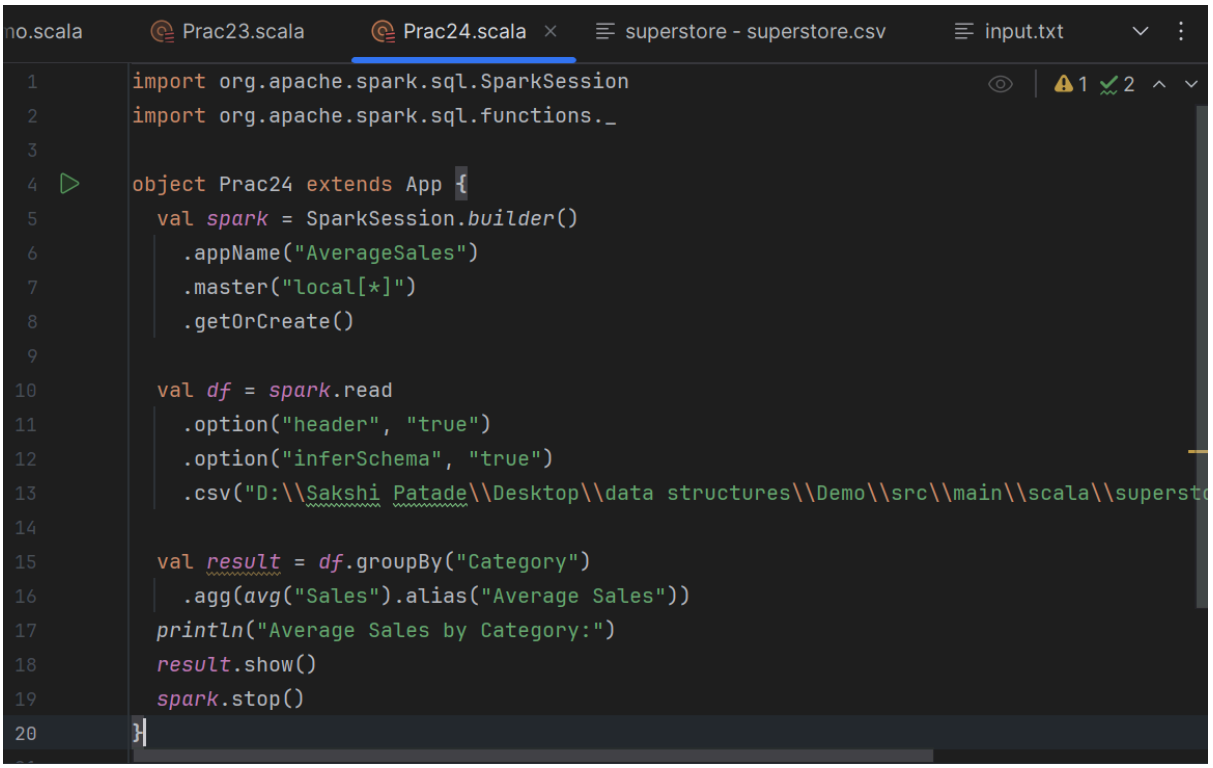
```scala
import org.apache.spark.sql.SparkSession

object Prac23 extends App {
  val spark = SparkSession.builder()
    .appName("FilterCSVExample")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val df = spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv("D:\\Sakshi Patade\\Desktop\\data structures\\Demo\\src\\main\\scala\\s

  val f = df.filter($"Sales" > 5000)

  println("Filtered Data:")
  f.show()

  spark.stop()
}
```

Output:

```
|Row ID|      Order ID|Order Date| Ship Date|     Ship Mode|Customer ID|       Customer Name|    Segment|       Country|         City|
+------+--------------+----------+----------+--------------+-----------+--------------------+-----------+--------------+-------------+
|   166|CA-2014-139892|  9/8/2014| 9/12/2014|Standard Class|   BM-11140|        Becky Martin|   Consumer|United States|  San Antonio|
|   510|CA-2015-145352| 3/16/2015| 3/22/2015|Standard Class|   CM-12385|Christopher Martinez|   Consumer|United States|      Atlanta|
|   684|US-2017-168116| 11/4/2017| 11/4/2017|      Same Day|   GT-14635|      Grant Thornton|  Corporate|United States|   Burlington|
|  2506|CA-2014-143917| 7/25/2014| 7/27/2014|  Second Class|   KL-16645|        Ken Lonsdale|   Consumer|United States|San Francisco|
|  2624|CA-2017-127180|10/22/2017|10/24/2017|   First Class|   TA-21385|        Tom Ashbrook|Home Office|United States|New York City|
|  2698|CA-2014-145317| 3/18/2014| 3/23/2014|Standard Class|   SM-20320|         Sean Miller|Home Office|United States| Jacksonville|
|  4099|CA-2014-116904| 9/23/2014| 9/28/2014|Standard Class|   SC-20095|        Sanjit Chand|   Consumer|United States|  Minneapolis|
|  4191|CA-2017-166709|11/17/2017|11/22/2017|Standard Class|   HL-15040|        Hunter Lopez|   Consumer|United States|       Newark|
|  4278|US-2016-107440| 4/16/2016| 4/20/2016|Standard Class|   BS-11365|        Bill Shonely|  Corporate|United States|     Lakewood|
|  6341|CA-2017-143112| 10/5/2017| 10/9/2017|Standard Class|   TS-21370|        Todd Sumrall|  Corporate|United States|New York City|
|  6426|CA-2016-143714| 5/23/2016| 5/27/2016|Standard Class|   CC-12370|  Christopher Conant|   Consumer|United States| Philadelphia|
|  6521|CA-2017-138289| 1/16/2017| 1/18/2017|  Second Class|   AR-10540|         Andy Reiter|   Consumer|United States|      Jackson|
|  6627|CA-2014-145541|12/14/2014|12/21/2014|Standard Class|   TB-21400|    Tom Boeckenhauer|   Consumer|United States|New York City|
|  6827|CA-2016-118689| 10/2/2016| 10/9/2016|Standard Class|   TC-20980|        Tamara Chand|  Corporate|United States|    Lafayette|
|  7667|US-2016-140158| 10/4/2016| 10/8/2016|Standard Class|   DR-12940|       Daniel Raglin|Home Office|United States|   Providence|
|  8154|CA-2017-140151| 3/23/2017| 3/25/2017|   First Class|   RB-19360|        Raymond Buch|   Consumer|United States|      Seattle|
```

Practical 24

Perform a group by operation in spark dataframes to compute the average of each group.



Ouptut:

Category	Average Sales
Office Supplies	121.69225531914947
Furniture	353.4459311957568
Technology	454.5405475802047